Chapter 8

Application Layer & Microservices

Why this chapter matters: so far we scaled *copies* of one application. Real products grow into dozens of features owned by many teams, and one deployable unit becomes a bottleneck for both engineering and operations. This chapter is about **how to split a system into layers and services**, how those services **find and talk to each other**, how to **keep one failure from taking everything down**, and — equally important — **when not to split at all**.

> ⚡ **QUICK RECALL — Chapter map**
> • 8.1 Layers: web vs application tier → 8.2 Monolith vs microservices → 8.3 Decomposition & data ownership
> • 8.4 Service-to-service communication → 8.5 Service discovery → 8.6 API gateway → 8.7 Service mesh
> • 8.8 Resilience patterns → 8.9 Data consistency: saga, outbox, CQRS → 8.10 Async workers
> • 8.11 Deployment & observability → 8.12 Disadvantages → 8.13 Worked example → 8.14 Practice, Q&A, cheat sheet

8.1 Separating the web layer from the application layer

The classic scalable architecture has three tiers. Separating out the **web layer** (reverse proxies, static content, request handling) from the **application layer** — also known as the **platform layer** (business logic, APIs) — lets you **scale and configure both layers independently**.

```
Request -> Load balancer -> Web servers (xN) -> Application servers (xM) -> Databases
                            reverse proxy,       business logic,           replicas, caches,
                            TLS, static files    APIs, workers             queues
Scale N and M independently: more traffic -> add web servers; heavy logic / new API -> add app servers.
```

- **Adding a new API results in adding application servers** without necessarily adding additional web servers. Heavy computation scales the app tier; heavy static/TLS traffic scales the web tier.
- Each tier can use its own hardware profile, runtime, security zone (web tier in a DMZ/public subnet, app tier private — Chapter 3) and release cycle.
- The **single responsibility principle** advocates small, autonomous services that work together. **Small teams with small services can plan more aggressively for rapid growth.**
- **Workers in the application layer also help enable asynchronism** — slow jobs move to queues and background workers (8.10, Chapter 12).

8.2 Monolith vs modular monolith vs microservices

Microservices are a suite of **independently deployable, small, modular services**. Each service runs a unique process and communicates through a well-defined, lightweight mechanism (HTTP/REST, gRPC, messages) to serve a business goal. Example (Pinterest-style): user profile, follower, feed, search, photo upload — each its own service.

	Monolith	Modular monolith	Microservices
Deploy unit	One application	One application, strict internal module boundaries	Many independently deployed services
Communication	In-process function calls (ns)	In-process calls via module interfaces	**Network calls** (ms, can fail) — Chapters 3–4
Data	One shared database	One DB, schema ownership per module	**Database per service**
Scaling	Scale the whole app	Scale the whole app	Scale each service independently
Team fit	Small team / early product	Medium team, clear domains	Many teams; autonomy matters

	Monolith	Modular monolith	Microservices
Complexity	Low ops, risk of tangled code	Low ops, disciplined code	High ops & distributed-systems complexity
Failure isolation	One bug or leak can take down all	Same	Contained to a service (if designed well)

What microservices buy you — and what they cost

Benefits	Costs
Independent deployment: small, frequent, low-risk releases per team **Independent scaling**: scale only the hot service (search, not profile) **Fault isolation** and technology freedom per service **Team autonomy**: ownership aligned with business domains (Conway's law) Smaller code bases that are easier to understand and rewrite	A different approach is needed **architecturally, operationally and process-wise** versus a monolith More complexity in deployments and operations: dozens of pipelines, configs, dashboards **Network is now in every call**: latency, timeouts, partial failure, retries **Data is distributed**: no joins or ACID across services (8.9) Harder testing, debugging and versioning; need tracing and good tooling

◆ **SENIOR INSIGHT** — **Monolith first, split on evidence**

- Most successful microservice systems started as a monolith that was split later, once domain boundaries were understood. A **modular monolith** gives most of the organisational benefit with none of the distributed-systems cost; extract a service only when there is a concrete reason.
- **Good reasons to split**: a component needs very different scaling or hardware (video transcoding, search); separate teams are blocked by a shared release train; a part needs different availability/security (payments); high rate of change in one area; a component can be reused by many.
- **Bad reasons**: 'it's modern', a tiny team, unclear domain boundaries, hoping to fix a messy codebase by adding a network.
- **Conway's law**: systems mirror the communication structure of the organisation that builds them. Design teams and services together (the 'inverse Conway manoeuvre').

✖ **COMMON MISTAKE** — **The distributed monolith**

- Services that must be deployed together, share one database, call each other in long synchronous chains, or break when another team ships — you get **all the costs of microservices and none of the benefits**. Warning signs: lock-step releases, cross-service transactions, shared tables, a single 'god' service everything depends on.

The eight fallacies of distributed computing (reasons every remote call differs from a local one)

The network is reliable • latency is zero • bandwidth is infinite • the network is secure • topology doesn't change • there is one administrator • transport cost is zero • the network is homogeneous. Each microservice call violates at least one of these, which is why timeouts, retries, idempotency and observability are mandatory (Chapter 4).

8.3 Decomposition & data ownership

Split by **business capability** (what the business does), not by technical layer. Domain-Driven Design calls the consistent boundaries **bounded contexts**: inside one, a word like "Customer" has one meaning; across contexts it may differ (Billing's customer ≠ Support's customer).

Service	Owns (its own data)	Typical store	Talks to
User / Profile	Accounts, profiles, settings	Relational (PostgreSQL)	Auth, Notification
Catalog	Products, prices, descriptions	Document DB + search index	Search, Inventory
Cart	Active carts	Redis / key-value (AP, Chapter 5)	Catalog, Pricing

Service	Owns (its own data)	Typical store	Talks to
Order	Orders, status history	Relational (strong consistency)	Payment, Inventory, Shipping (events)
Payment	Transactions, ledgers	Relational, ACID, audited	External PSP, Order
Inventory	Stock levels, reservations	Relational with row locks / counters	Order, Catalog
Notification	Templates, delivery logs	Queue + wide-column/document	Email/SMS/push providers

★ **MUST REMEMBER** — **Database per service**

- Each service owns its data and exposes it **only through its API or events**. No other service reads or writes its tables directly. This preserves independent deployment and lets each service choose the best database (polyglot persistence), at the price of no cross-service joins/transactions (8.9).

Sizing heuristics: a service should be owned by **one team** (the 'two-pizza team'), be understandable in days, replaceable in weeks, change for one reason, and rarely need to be deployed together with another service. Avoid **nano-services** (so small that every request fans out across ten of them), **shared databases**, **chatty** service pairs (merge them), and **circular dependencies**.

◆ **SENIOR INSIGHT** — **Migrating a monolith: the Strangler Fig pattern**

- Put a routing layer (gateway/proxy) in front of the monolith. Build one capability as a new service, **route that slice of traffic** to it (gradually, with canary), keep the rest in the monolith, and repeat until the monolith withers. Use an **anti-corruption layer** so the new model isn't polluted by legacy concepts. Avoid 'big-bang rewrites'.

8.4 Service-to-service communication

Style	How	Pros	Cons
Synchronous (REST, gRPC)	Caller waits for the response	Simple mental model; immediate result; easy request/response	Temporal coupling: callee must be up; latency adds up; availability multiplies (series, Chapter 5)
Asynchronous (events / queues)	Caller publishes a message/event and moves on; consumers react	Loose coupling, buffering of spikes, easy fan-out, resilience to downstream outages	Eventual consistency, harder to trace and debug, duplicates → idempotent consumers

Commands ("charge this card") go to one service and expect action; **events** ("OrderPlaced") state a fact and may be consumed by many. Prefer events for notifying, commands for requesting.

Choreography vs orchestration

	Choreography	Orchestration
Idea	No central brain; each service reacts to events and emits new events	A central **orchestrator** (workflow engine / saga coordinator) tells each service what to do next
Pros	Loose coupling, no single coordinator, easy to add subscribers	Flow is explicit and visible in one place; easier to reason about, retry and monitor
Cons	Business flow is scattered; hard to see the whole process; risk of cyclic event chains	Orchestrator can become a bottleneck / god service; more coupling to it
Use when	Simple flows, few steps, many independent reactions	Complex multi-step business processes with compensation (order fulfilment); tools: Temporal, AWS Step Functions, Camunda

◆ **SENIOR INSIGHT** — **Keep synchronous chains short**

- Depth hurts twice: **availability** multiplies (6 synchronous services at 99.9% each = $0.999^6 \approx 99.4\%$, about 53 hours of downtime/year) and **tail latency** compounds (if each hop has a 1% chance of being slow, five hops give $1 - 0.99^5 \approx 4.9\%$ of requests hit at least one slow hop).
- Techniques: move non-critical work off the request path (events), **API composition** in the gateway/BFF for read aggregation, caching, parallel calls instead of sequential, and denormalised read models (8.9).

8.5 Service discovery

In a cloud environment instances come and go (autoscaling, deployments, failures) and get new IP addresses. Hard-coding addresses does not work. **Service discovery** keeps track of **registered names, addresses and ports**, so services can find each other. Systems such as **Consul, Etcd and Zookeeper** help with this.

```
1) REGISTER + HEARTBEAT                    SERVICE REGISTRY
   orders @ 10.0.1.5:8080  ------------>   (Consul / etcd / ZooKeeper / Eureka / Kubernetes API)
   orders @ 10.0.1.9:8080  ------------>   orders -> [10.0.1.5:8080, 10.0.1.9:8080]  (healthy only)

2) LOOKUP                                          3) CALL
   Client service B: "where is orders?" --> registry --> list of healthy addresses
      client-side discovery : B picks one instance itself (load balances in the client)
      server-side discovery : B calls a router / load balancer / K8s Service, which consults the registry
```

	Client-side discovery	Server-side discovery
Who looks up the registry?	The calling service (library: Eureka client, gRPC resolver)	A router / load balancer / platform (AWS ALB with target groups, **Kubernetes Service**)
Pros	No extra hop; client can use smart load balancing	Clients stay simple and language-agnostic
Cons	Discovery logic in every client language	Extra component to run and keep highly available

- **Registration**: *self-registration* (the instance registers itself and sends heartbeats) or *third-party registration* (the platform/orchestrator registers it).
- **Health checks help verify service integrity** and are often done using an **HTTP endpoint** (`/health`). Unhealthy or silent instances are deregistered after a TTL (Chapter 7).
- **Both Consul and Etcd have a built-in key-value store** that can be useful for storing config values and other shared data (feature flags, leader locks).
- **Consistency choices** (Chapter 5): ZooKeeper, etcd and Consul use consensus (ZAB/Raft) → CP; Netflix Eureka favours availability (AP) and keeps serving possibly stale data during partitions. Clients must therefore tolerate stale entries: connect timeouts, retries on another instance, and outlier ejection.
- **DNS-based discovery**: SRV/A records, with the TTL trade-offs of Chapter 6. In Kubernetes every Service gets a stable name such as `orders.shop.svc.cluster.local` and a virtual IP that balances to healthy pods.

✎ **CORRECTION TO YOUR ORIGINAL NOTE** — **Where Consul/etcd/ZooKeeper fit today**

- On **Kubernetes**, discovery is built in (Services + DNS + readiness probes), so you rarely run Consul just for discovery; **etcd** is the cluster's own state store, not something applications should abuse. ZooKeeper/etcd/Consul remain valuable for **coordination**: leader election, distributed locks, configuration and multi-platform (VM + container) environments.

8.6 API gateway

An **API gateway** is the single entry point for external clients. It receives every request, applies cross-cutting policies, and routes (or composes) calls to internal services, so clients don't need to know the internal topology (it extends the reverse-proxy idea from Chapter 7).

Responsibility	Details
Routing & composition	/users → user-service, /orders → order-service; aggregate several backend calls into one response
Authentication & authorisation	Validate JWT/OAuth tokens or API keys once at the edge; pass identity downstream
Rate limiting & quotas	Per client/key/IP; protect backends (Chapter 14)
TLS termination, CORS, compression, caching	Same offloads as a reverse proxy
Protocol translation	REST/GraphQL for clients ↔ gRPC internally; WebSocket upgrades
Observability	Central access logs, metrics, request ids / trace context injection
Versioning & canary routing	/v1 vs /v2, header- or percentage-based traffic splits

◆ **SENIOR INSIGHT** — **Patterns and pitfalls**

- **Backend For Frontend (BFF)**: a separate gateway per client type (web, iOS, Android, partners) shaped to that client's needs — reduces over-fetching and round trips (Chapter 4, GraphQL).
- **Risks**: the gateway is a **single point of failure and a bottleneck** (run it redundantly and autoscaled, Chapter 7); keep it **thin** — business logic in the gateway recreates a monolith; avoid it becoming a team bottleneck for every change.
- Examples: Kong, Apigee, AWS API Gateway, Azure API Management, Envoy/NGINX-based gateways, Spring Cloud Gateway.

8.7 Service mesh

A **service mesh** moves service-to-service networking concerns out of application code and into a **sidecar proxy** (usually Envoy) deployed next to every service instance. A central **control plane** (Istio, Linkerd, Consul Connect) configures all the proxies; the proxies are the **data plane**.

```
[ Service A + sidecar ] <===== mTLS, retries, timeouts, metrics =====> [ sidecar + Service B ]
          ^                                                                      ^
          +----------- CONTROL PLANE (Istio, Linkerd, Consul): config, certs, policy -----+
Data plane = all sidecar proxies (Envoy).  Control plane = pushes routing, certs, policy; collects telemetry.
```

Mesh provides (without changing app code)	Costs / when to avoid
mTLS between all services (identity + encryption) Retries, timeouts, **circuit breaking**, outlier detection Smart load balancing (P2C, locality), **traffic shifting** (canary 5% → 100%), fault injection Uniform metrics, logs and distributed traces (golden signals) Policy: who may call whom (zero trust)	Extra hop per call (sub-millisecond to a few ms) and CPU/memory per sidecar Significant operational complexity and a steep learning curve Debugging gets harder (another layer) Overkill for a handful of services — a shared client library or gateway may be enough Newer designs reduce overhead (sidecar-less / ambient, eBPF)

Rule of thumb: gateway = *north–south* traffic (clients ↔ cluster); service mesh = *east–west* traffic (service ↔ service).

8.8 Resilience patterns (preventing cascading failure)

One slow dependency can exhaust threads and connections in its callers, then in their callers — a **cascading failure**. Combine these defences (timeouts, retries and idempotency are in Chapter 4):

Pattern	What it does	Example
Timeout + deadline propagation	Never wait forever; pass the remaining time budget downstream	Call timeout 800 ms inside a 2 s user-facing deadline

Pattern	What it does	Example
Retry with backoff + jitter, budgets	Recover from transient errors without a retry storm	Max 2 retries, only idempotent calls
Circuit breaker	After repeated failures stop calling the sick dependency (fail fast), then probe to recover	States below
Bulkhead	Isolate resource pools (threads, connections) per dependency so one failure can't drain everything	Separate pool for the recommendation service
Fallback / graceful degradation	Return a cached, default or partial result	Show generic recommendations if the personaliser is down
Load shedding & rate limiting	Reject excess work early to stay healthy (503/429)	Drop low-priority traffic at 90% CPU
Backpressure	Slow producers when consumers can't keep up (bounded queues)	Block or reject when the queue is full
Health checks & autoscaling	Remove bad instances, add capacity	Readiness probes, HPA

Circuit breaker states

```
  CLOSED    normal operation, count failures
     |  failures >= threshold
     v
  OPEN      fail fast: no calls reach the dependency
     |  cooldown elapsed
     v
  HALF-OPEN allow a few trial calls
     |  trial succeeds -> back to CLOSED        trial fails -> back to OPEN
```

```
class CircuitBreaker {
  constructor(fn, { threshold = 5, cooldownMs = 10_000 } = {}) {
    Object.assign(this, { fn, threshold, cooldownMs, failures: 0, state: 'CLOSED', openedAt: 0 });
  }
  async call(...args) {
    if (this.state === 'OPEN') {
      if (Date.now() - this.openedAt < this.cooldownMs) throw new Error('circuit open: failing fast');
      this.state = 'HALF_OPEN';                          // cooldown over: allow a trial request
    }
    try {
      const result = await this.fn(...args);
      this.failures = 0; this.state = 'CLOSED';          // success closes the circuit
      return result;
    } catch (err) {
      if (this.state === 'HALF_OPEN' || ++this.failures >= this.threshold) {
        this.state = 'OPEN'; this.openedAt = Date.now(); // trip the breaker
      }
      throw err;
    }
  }
}
```

8.9 Data consistency across services: saga, outbox, CQRS

With a database per service you lose cross-service ACID transactions and joins. Three tools replace them:

1) Saga — a sequence of local transactions with compensations

A saga splits a business transaction (place an order) into local transactions, each in its own service, that publish events. If a step fails, **compensating transactions** undo the earlier steps (semantic rollback). Sagas give **eventual consistency** (Chapter 5), not isolation: other readers may see intermediate states.

Step	Service action	Compensation if a later step fails
1	Order: create order in state PENDING	Mark order CANCELLED
2	Payment: authorise/charge the card	Refund / void the authorisation
3	Inventory: reserve items	Release the reservation
4	Shipping: schedule shipment	Cancel shipment
5	Order: mark CONFIRMED	—

If step 3 fails (out of stock): run compensations in reverse — refund payment (2), cancel order (1) — and tell the customer. Orchestrated sagas keep this flow in one coordinator; choreographed sagas react to events (8.4). Every step and compensation must be **idempotent** and retryable.

> ◆ **SENIOR INSIGHT** — **Why not two-phase commit (2PC)?**
> - 2PC makes a coordinator ask all participants to 'prepare' then 'commit'. It gives atomicity but **blocks** participants holding locks while waiting, the coordinator is a SPOF, latency is high, and many modern stores/brokers don't support it. In microservices at scale it harms availability (CAP, Chapter 5) — sagas trade strict atomicity for availability.

2) Transactional outbox — reliably publish events

Problem: updating the DB *and* publishing an event to a broker are two operations; a crash between them loses the event or publishes a lie (the dual-write problem). Solution: write the business row **and** an *outbox* row in the **same local DB transaction**; a relay (poller or CDC such as Debezium) publishes outbox rows to the broker and marks them sent. Delivery is **at-least-once**, so consumers must be **idempotent** (dedupe by event id).

3) CQRS and materialised read models

Separate the **write model** (commands, normalised, strongly consistent) from **read models** built from events and optimised for queries — e.g., an 'order history' view that joins data from several services into one denormalised table or search index. Reads become fast and independent of other services; the cost is eventual consistency and extra storage. **Event sourcing** (store the log of events as the source of truth) pairs naturally with CQRS but adds significant complexity — adopt it only where an audit trail or replay is a real need.

8.10 Asynchronous workers in the application layer

Workers in the application layer help enable asynchronism: the web tier accepts a request, places a job on a queue and replies quickly; a pool of workers processes jobs in the background (Chapter 12 goes deep).

```
1) Request -> Web/API server: validate, enqueue job ------> [ Message queue ]
2) Web/API server replies at once: 202 Accepted + job id       |
3) Worker pool pulls jobs  <-----------------------------------+
   (resize image / send email / generate report)
4) Worker updates job status; user is notified (webhook / SSE / polling the job id)
```

- **Scale workers independently** of the web tier, based on **queue depth / age of oldest message**, not CPU alone.
- Use for: email/SMS, image/video processing, report generation, search indexing, third-party calls, cache warming, fan-out of feeds.
- Make jobs **idempotent**, retry with backoff, send failures to a **dead-letter queue**, and keep job payloads small (pass ids, not blobs).

8.11 Deployment, operations & observability

Area	What good looks like
Packaging & runtime	One container image per service; Kubernetes (Deployment → ReplicaSet → Pods; Service for stable address; Ingress/Gateway for entry; HPA for autoscaling; ConfigMap/Secret for config)
CI/CD	Independent pipeline per service: build → test → scan → deploy; **rolling**, **blue-green** or **canary** releases with automatic rollback; feature flags to decouple deploy from release
Compatibility	Backward-compatible API/schema changes (add, don't break); consumer-driven **contract tests** so teams can release without end-to-end test marathons
Configuration & secrets	Externalised config per environment; secrets in a vault, never in images or repos; same artifact promoted across environments
Observability	Three pillars: **logs** (structured, with correlation/trace id), **metrics** (RED: rate, errors, duration; USE: utilisation, saturation, errors), **distributed traces** (OpenTelemetry → Jaeger/Tempo). Alert on SLOs and burn rate (Chapter 5)
Ownership	'You build it, you run it': the owning team carries the pager, owns the SLOs, runbooks and on-call

★ **MUST REMEMBER — Distributed tracing is non-negotiable**

- Generate a **trace id** at the edge (gateway) and propagate it in headers (`traceparent`) through every call, queue message and log line. Without it, finding which of 15 services made a request slow is guesswork.

8.12 Disadvantages (summary)

Disadvantage	Detail	Mitigation
Different architecture, operations and process	Adding an application layer with loosely coupled services requires a different approach from an architectural, operations and process viewpoint (versus a monolithic system)	Platform team, paved roads (templates, CI/CD, observability by default), training
Deployment & operations complexity	Microservices can add complexity in terms of deployments and operations	Containers + Kubernetes, IaC, GitOps, standard pipelines
Network failures & latency	Every call may be slow or fail; chains compound	Resilience patterns (8.8), shorter chains, async
Data consistency	No cross-service transactions or joins	Sagas, outbox, CQRS; accept eventual consistency where possible
Testing & debugging	Integration across services; non-local failures	Contract tests, tracing, staging with realistic data, chaos testing
Versioning & coupling	Breaking changes ripple across teams	Compatible evolution, deprecation policy, schema registry
Cost	More infrastructure, duplicated tooling, more people on platform work	Right-size; consolidate services that always change together

8.13 Worked example: designing an e-commerce checkout

Goal: place an order. Which calls must be synchronous (user waits) and which can be asynchronous?

Step	Call	Sync or async?	Reason
1	Cart → Catalog: validate items and prices	**Sync**	Must be correct before charging
2	Order → Inventory: reserve stock	**Sync**	Prevent overselling (strong consistency on the decrement)

Step	Call	Sync or async?	Reason
3	Order → Payment: authorise the card	**Sync**	User needs an immediate success/failure
4	Order: persist CONFIRMED + write outbox event	Local transaction	Atomic with the event
5	Shipping, email/SMS, loyalty points, analytics, recommendations	**Async** (events)	Not needed to answer the user; failures retried independently

```
Critical sync path: 3 services at 99.9% each      -> 0.999^3 = 99.70%   (~26 h downtime/yr)
If we had made all 6 steps synchronous            -> 0.999^6 = 99.40%   (~53 h downtime/yr)
Moving shipping/email/analytics to events removes 3 services from the availability product,
and a notification outage no longer blocks a purchase.
```

Failure walkthrough: payment succeeds, inventory reservation times out. The order service retries with the same idempotency key (Chapter 4); if it still fails, the saga runs compensations — void the payment authorisation, mark the order FAILED — and the user sees a clear message. A 'PaymentVoided' event lets the notification service email them. Each step is traced with one trace id.

8.14 Practice questions (try before looking)

P1. Your team of 6 is building a new product with unclear requirements. Microservices or monolith?

Answer: A (modular) **monolith**. Domain boundaries are unknown, the team is small, and distributed costs would slow you down. Split later along proven seams.

P2. Service A synchronously calls B, C, D, E, each 99.95% available, and A is 99.95%. Overall availability?

Answer: $0.9995^5 \approx 0.99750$ → **99.75%** (~22 hours downtime/year). Reduce by making some calls asynchronous or cached.

P3. Orders and Inventory are separate services with separate databases. How do you place an order atomically?

Answer: No distributed ACID. Use a **saga**: reserve inventory → charge payment → confirm order, with compensations (release stock, refund) on failure; make each step idempotent; use the outbox pattern for reliable events.

P4. A downstream service becomes slow (10 s per call). What protects your service from running out of threads?

Answer: Timeouts, a **circuit breaker** to fail fast, a **bulkhead** (separate pool for that dependency), fallbacks and load shedding.

P5. When is a service mesh worth it, and when is an API gateway enough?

Answer: Mesh: many (dozens+) services, polyglot, need uniform mTLS, retries, traffic shifting and telemetry without code changes. Gateway: north–south concerns for external clients; with few services, shared libraries + gateway suffice.

Interview Q&A (30-second answers)

Q1. Monolith vs microservices — how do you decide?

Start with a (modular) monolith; split when you have independent scaling needs, team/release bottlenecks, differing availability or security needs, and clear bounded contexts. Microservices trade code complexity for operational and distributed-systems complexity.

Q2. What is a distributed monolith?

Services that are tightly coupled (shared DB, lock-step deploys, long sync chains) — all the costs of microservices, none of the independence.

Q3. How do services find each other?

Service discovery: a registry (Consul, etcd, ZooKeeper, Eureka, or Kubernetes DNS/Services) tracks names, addresses and health; discovery is client-side or server-side.

Q4. What does an API gateway do vs a service mesh?

Gateway handles external (north–south) traffic: routing, auth, rate limiting, aggregation. Mesh handles internal (east–west) traffic via sidecars: mTLS, retries, circuit breaking, telemetry.

Q5. How do you handle a transaction across services?

Saga with compensating actions (orchestrated or choreographed), idempotent steps, transactional outbox for reliable events; avoid 2PC because it blocks and hurts availability.

Q6. Database per service — why and how do you do joins?

Ownership and independent deployment. Joins become API composition, events with local read models (CQRS), or search indexes; accept eventual consistency.

Q7. Choreography vs orchestration?

Choreography: services react to events (loose coupling, scattered flow). Orchestration: central coordinator drives steps (explicit flow, easier error handling, coupling to coordinator).

Q8. How do you prevent cascading failures?

Timeouts, retries with backoff+jitter and budgets, circuit breakers, bulkheads, fallbacks, load shedding, backpressure, and capacity headroom.

Q9. What is the transactional outbox pattern?

Write the state change and the event to an outbox table in one local transaction; a relay publishes it to the broker with at-least-once delivery; consumers dedupe.

Q10. How do you migrate from a monolith?

Strangler fig: route slices of traffic to new services behind a gateway, extract capabilities one by one, use anti-corruption layers, never a big-bang rewrite.

AMJESH https://singhamjesh.github.io/

Chapter 8 Cheat Sheet

⚡ **QUICK RECALL** — **Revise this in 2 minutes**

- **Layers:** separate web tier from application (platform) tier → scale/configure independently; new API ⇒ add app servers. Workers enable asynchronism.
- **Microservices** = small, independently deployable services with their own process and data, talking via lightweight protocols. **Monolith first**; split on evidence (scaling, teams, availability).
- **Costs:** network in every call, distributed data, ops/testing/observability complexity. Avoid the **distributed monolith**. Remember the 8 fallacies.
- **Decompose** by business capability (bounded context); **database per service**; no shared tables; avoid nano-services and chatty pairs. Migrate with **strangler fig**.
- **Communication:** sync (REST/gRPC) vs async (events/queues); choreography vs orchestration; keep sync chains short (availability multiplies).
- **Service discovery:** registry (Consul / etcd / ZooKeeper / Eureka / K8s DNS) with health checks; client-side vs server-side; KV store for config; CP vs AP registries.
- **API gateway** = north–south: routing, auth, rate limit, aggregation, BFF; keep it thin and redundant. **Service mesh** = east–west via sidecars: mTLS, retries, traffic shifting, telemetry.
- **Resilience:** timeout, retry+jitter, circuit breaker (closed/open/half-open), bulkhead, fallback, load shedding, backpressure.
- **Consistency: saga** (local transactions + compensations), **outbox** (atomic state+event, at-least-once, idempotent consumers), **CQRS** read models; avoid 2PC.
- **Operate:** containers + Kubernetes, per-service CI/CD, canary/blue-green, contract tests, logs+metrics+traces with a trace id, SLOs, you-build-it-you-run-it.

AMJESH https://singhamjesh.github.io/